

12/07/2026

# Heart Disease Risk Prediction

## End-to-End MLOps Pipeline

**Course:** AIMLCZG523 - Machine Learning Operations

**Author:** Adithya M Sasi

**Student ID:** 2024AC05785

**Repository:** [github.com/axle-bits/MLOP\\_Assignment\\_1](https://github.com/axle-bits/MLOP_Assignment_1)

Video link: <https://drive.google.com/file/d/1BwhptAhkgGb4PhyiduOTC0kxJeVRczWQ/view?usp=sharing>

HQ video: [https://drive.google.com/file/d/1jHvD1lelUSfkQ5ww2sir\\_IIfRbP\\_ZC-R/view?usp=drive\\_link](https://drive.google.com/file/d/1jHvD1lelUSfkQ5ww2sir_IIfRbP_ZC-R/view?usp=drive_link)

Youtube link: [https://youtu.be/eqm2iqW20\\_s?si=EdjgnevJ\\_TtwBY9U](https://youtu.be/eqm2iqW20_s?si=EdjgnevJ_TtwBY9U)

## Contents

|                                             |    |
|---------------------------------------------|----|
| Contents.....                               | 1  |
| 1. Overview & Setup .....                   | 2  |
| Quickstart.....                             | 2  |
| 2. Data & EDA .....                         | 4  |
| 3. Feature Engineering & Models.....        | 7  |
| Derived Clinical Features .....             | 7  |
| Preprocessing.....                          | 7  |
| Training Setup.....                         | 7  |
| Results.....                                | 7  |
| 4. Experiment Tracking.....                 | 9  |
| MLflow 3.14 Adaptations .....               | 11 |
| 5. Packaging & Reproducibility.....         | 12 |
| 6. CI/CD Pipeline.....                      | 13 |
| 7. Containerisation.....                    | 15 |
| Image Design .....                          | 15 |
| Local Proof.....                            | 15 |
| API Input Validation .....                  | 16 |
| 8. Deployment.....                          | 17 |
| Kubernetes Manifests .....                  | 17 |
| Verification.....                           | 17 |
| 9. Monitoring & Logging .....               | 20 |
| Structured Logging.....                     | 20 |
| Prometheus Metrics.....                     | 20 |
| Grafana Dashboard .....                     | 20 |
| Observability Lesson: Scrape Topology ..... | 21 |
| 10. Conclusion.....                         | 22 |

## 1. Overview & Setup

This project builds an end-to-end MLOps pipeline around a binary classifier that predicts heart disease risk from 13 clinical features in the UCI Cleveland dataset. The goal was to evidence every stage of a production ML lifecycle: reproducible data preparation, tracked experimentation, automated testing and CI/CD, containerised serving, Kubernetes deployment, and monitoring. Every non-trivial choice along the way is recorded and justified, not assumed.

The repository is organised around a stage-oriented layout that maps 1:1 onto CI/CD stages:

- ml/ - data download, preprocessing, feature engineering, training, and export
- api/ - FastAPI serving code and Pydantic schemas
- infra/ - Dockerfile and Kubernetes manifests
- tests/ - 39 pytest unit and integration tests
- notebooks/ - jupyter-text-paired EDA, model-comparison, and inference notebooks
- docs/ - decision log, report, and figures

Architecture overview:

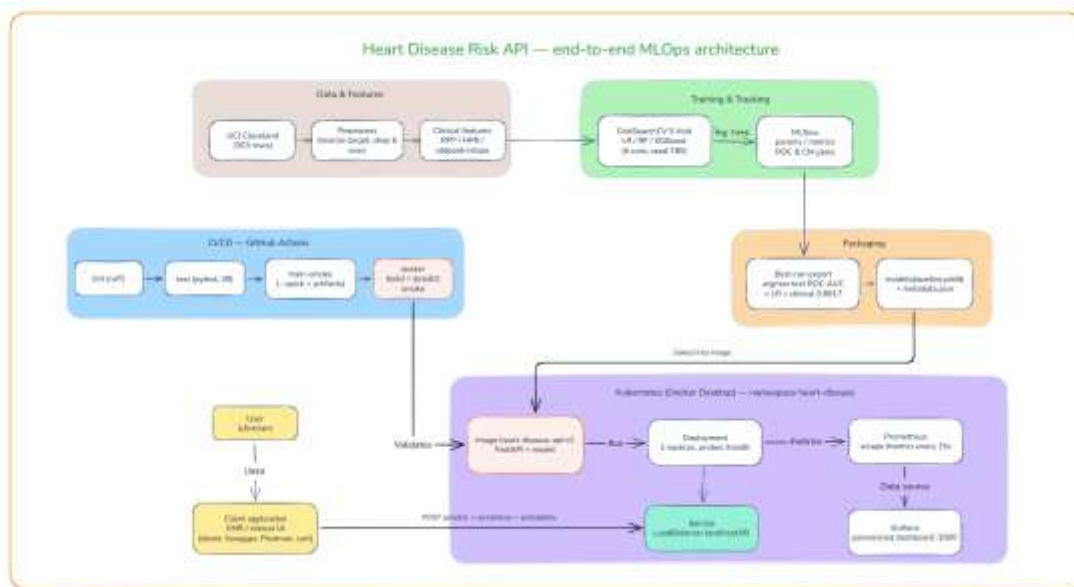


Figure 1 - System architecture

## Quickstart

```
python -m venv .venv && .venv/Scripts/activate
pip install -r requirements.txt
```

Train and track experiments, then serve locally:

```
python -m ml.models.train
uvicorn api.main:app --reload --port 8000
```

Or run containerised and on Kubernetes:

```
docker build -t heart-disease-api -f infra/Dockerfile .  
docker run -d -p 8000:8000 heart-disease-api  
kubectl apply -f infra/k8s/
```

Run the test suite with `pytest` and the lint gate with `ruff check` . before touching anything else, to confirm the committed pipeline is clean. The remaining sections follow the same order as the assignment's nine tasks.

## 2. Data & EDA

The dataset is the classic Cleveland subset of the UCI Heart Disease repository, fetched programmatically via `ucimlrepo (id=45)`. This returns exactly 303 rows  $\times$  13 features and matches the assignment's dataset link. The combined four-site dataset (~920 rows) was rejected because it is not what `id=45` serves.

The raw multi-class target num (0–4, distribution 164/55/36/35/13) was binarised to 0 (no disease) vs 1 (disease present), the established 0-vs-rest convention. Six rows (2% of data) with missing `ca` (4 rows) or `thal` (2 rows) values were dropped rather than imputed - both are categorical clinical measurements where imputation would invent clinical facts. This leaves 297 rows with a class balance of 160 no-disease vs 137 disease cases.

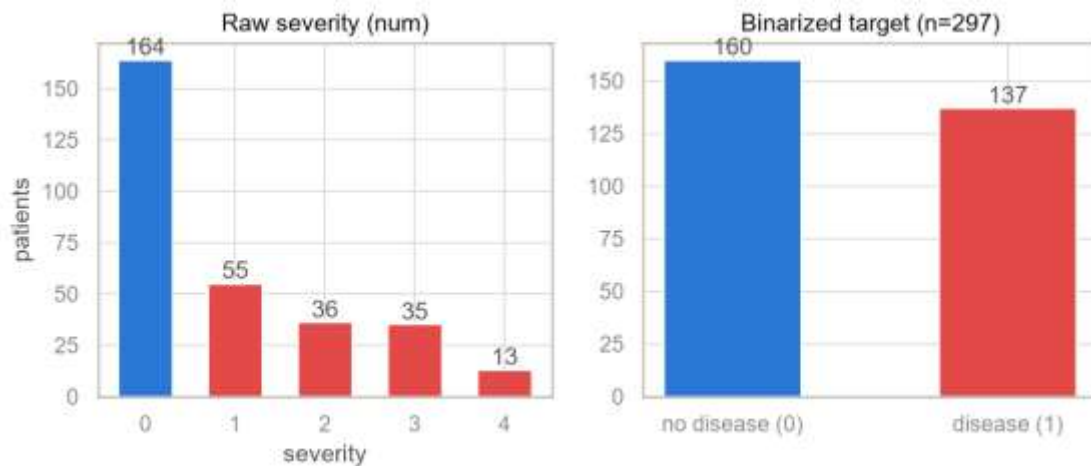


Figure 2 - Class balance (297 rows post-cleaning)

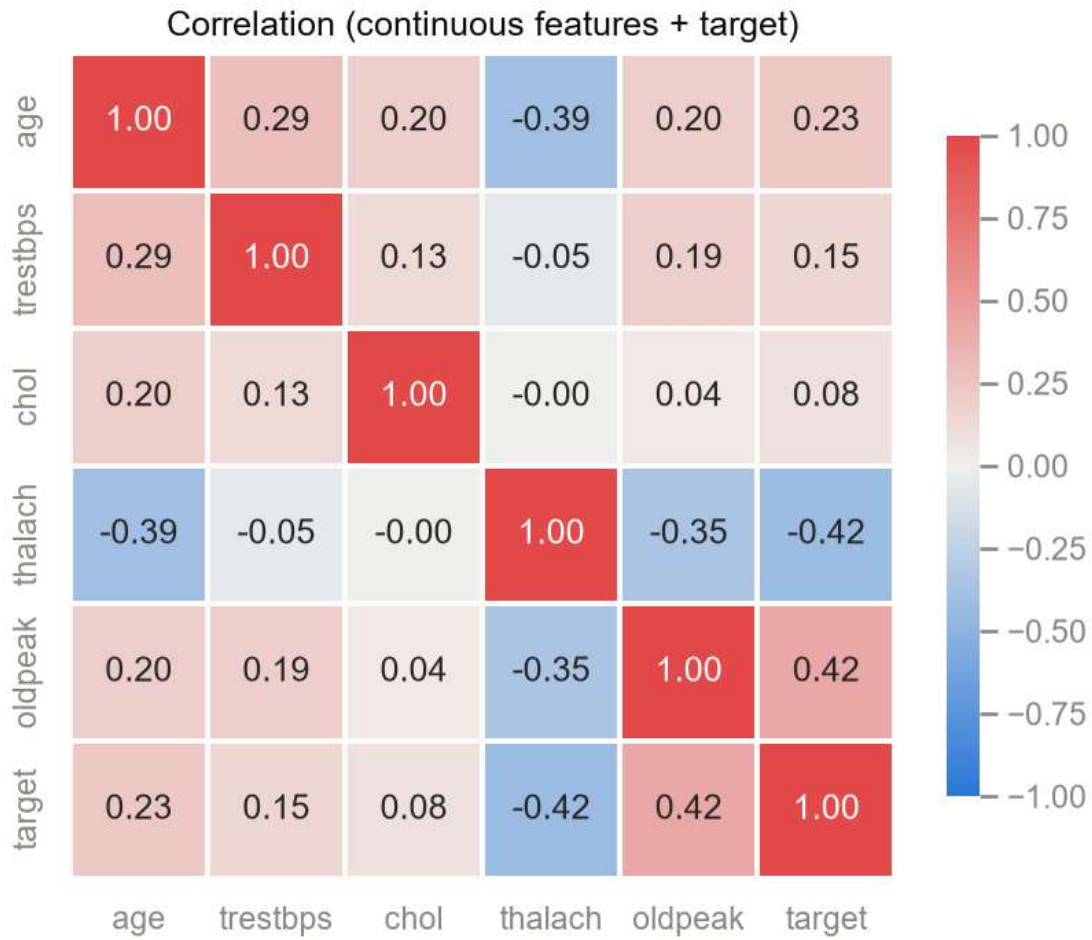


Figure 3 - Correlation heatmap (diverging blue-grey-red palette)

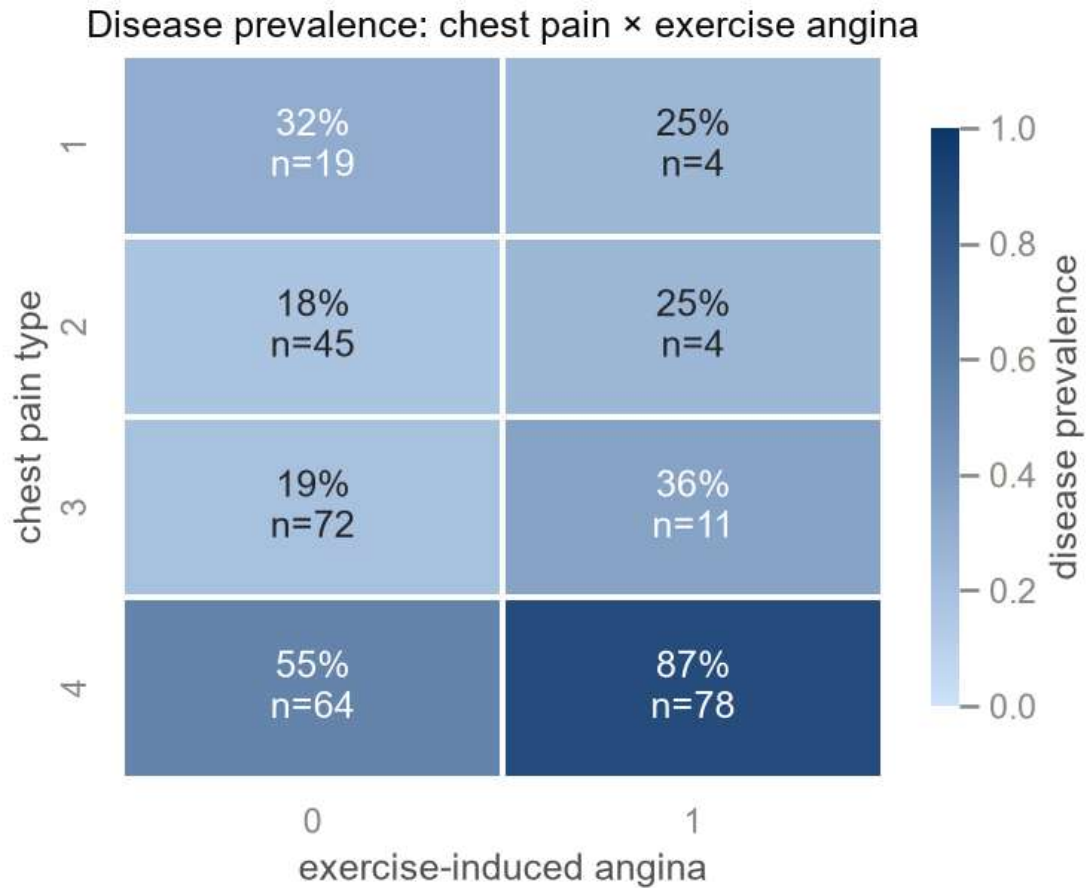


Figure 4 - Chest pain type × exercise angina prevalence heatmap

Key EDA findings: disease prevalence is higher in males (56%) vs females (26%). Patients with asymptomatic chest pain and exercise-induced angina show 87% prevalence - the strongest subgroup signal. ST-depression (oldpeak,  $r=+0.42$ ) and maximum heart rate (thalach,  $r=-0.42$ ) are the strongest continuous correlates of the target.

All figures use a fixed two-class palette (no-disease #2a78d6 blue, disease #e34948 red) validated colorblind-safe with a worst-pair  $\Delta E$  well above the accessibility threshold.

### 3. Feature Engineering & Models

#### Derived Clinical Features

Three clinically motivated features are computed inside the scikit-learn Pipeline (via FunctionTransformer), not pre-computed into the cleaned CSV:

- Rate-pressure product:  $\text{trestbps} \times \text{thalach}$  - index of myocardial oxygen demand
- Heart-rate reserve:  $220 - \text{age} - \text{thalach}$  - unused chronotropic capacity
- $\text{oldpeak} \times \text{slope}$  interaction - ST-depression magnitude  $\times$  slope pattern

Computing features inside the pipeline guarantees identical transformation at training and inference. A fourth candidate (pulse pressure) was rejected: the dataset carries only systolic BP (trestbps) with no diastolic column.

#### Preprocessing

The ColumnTransformer applies three strategies:

- StandardScaler  $\rightarrow$  5 continuous features + ca (ordinal vessel count 0–3) + 3 derived features
- OneHotEncoder(handle\_unknown='ignore')  $\rightarrow$  nominal categoricals: cp, restecg, slope, thal
- Passthrough  $\rightarrow$  binary flags: sex, fbs, exang

#### Training Setup

Three model families (Logistic Regression, Random Forest, XGBoost)  $\times$  2 feature sets (raw / raw + clinical) = 6 tracked runs. Every run used GridSearchCV over StratifiedKFold (5 folds, shuffle=True), scoring ROC-AUC, tuned on an 80/20 stratified split. random\_state=785 (derived from student ID 2024AC05785) is shared across all runs.

#### Results

| Model                      | Features        | CV ROC-AUC                         | Accuracy     | Precision    | Recall       | F1           | Test ROC-AUC   |
|----------------------------|-----------------|------------------------------------|--------------|--------------|--------------|--------------|----------------|
| <b>Logistic Regression</b> | <b>Clinical</b> | <b>92.0% <math>\pm</math> 1.23</b> | <b>76.7%</b> | <b>75.0%</b> | <b>75.0%</b> | <b>75.0%</b> | <b>★ 88.2%</b> |
| Logistic Regression        | Raw             | 92.0% $\pm$ 2.30                   | 78.3%        | 77.8%        | 75.0%        | 76.4%        | 87.6%          |
| Random Forest              | Raw             | 92.4% $\pm$ 1.49                   | 78.3%        | 75.9%        | 78.6%        | 77.2%        | 85.8%          |
| Random Forest              | Clinical        | 91.5% $\pm$ 2.06                   | 76.7%        | 79.2%        | 67.9%        | 73.1%        | 84.9%          |
| XGBoost                    | Clinical        | 93.1% $\pm$ 1.15                   | 68.3%        | 65.5%        | 67.9%        | 66.7%        | 84.2%          |
| XGBoost                    | Raw             | 92.4% $\pm$ 1.96                   | 75.0%        | 69.7%        | 82.1%        | 75.4%        | 83.2%          |

**KEY FINDING** Winner: Logistic Regression + Clinical features - Test ROC-AUC 88.2%, CV 92.0%  $\pm$  1.2%. The clinical features helped LR (+0.6pp AUC) but slightly hurt Random Forest (−0.9pp AUC, −10.7pp recall).



Tree ensembles already capture the interactions that the engineered features encode.

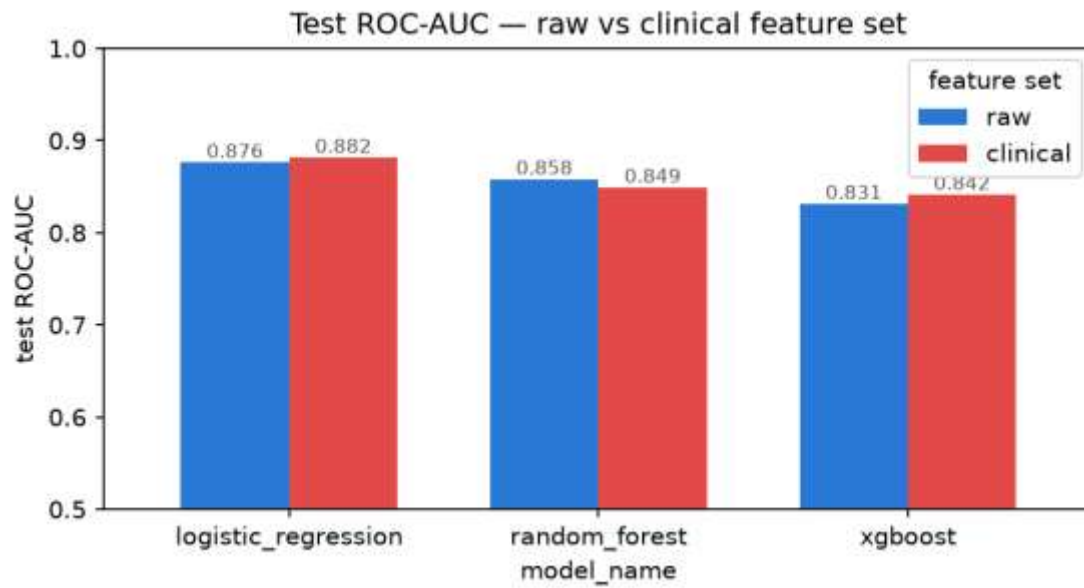


Figure 5 - Test ROC-AUC: raw vs clinical feature set per model

## 4. Experiment Tracking

All six runs are tracked with MLflow using a `sqlite:///mlflow.db` store for run metadata (params, metrics, tags) with artifacts under `./mlruns`. Each run logs:

- Hyperparameters (best GridSearchCV params, seed, CV fold count, quick\_mode flag)
- Seven metrics: CV mean/std ROC-AUC + test accuracy, precision, recall, F1, ROC-AUC
- Artifacts: ROC curve, confusion matrix, and feature-importance plots
- The fitted pipeline in MLflow model format (cloudpickle serialisation)



Figure 6 - MLflow experiment list

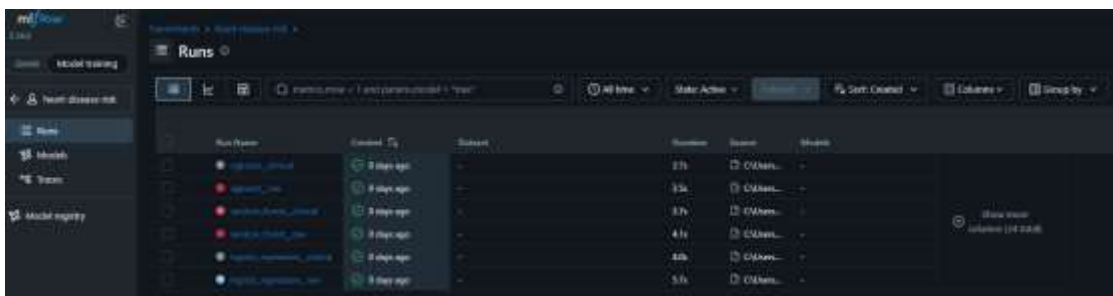


Figure 7 - All six runs in the MLflow runs table

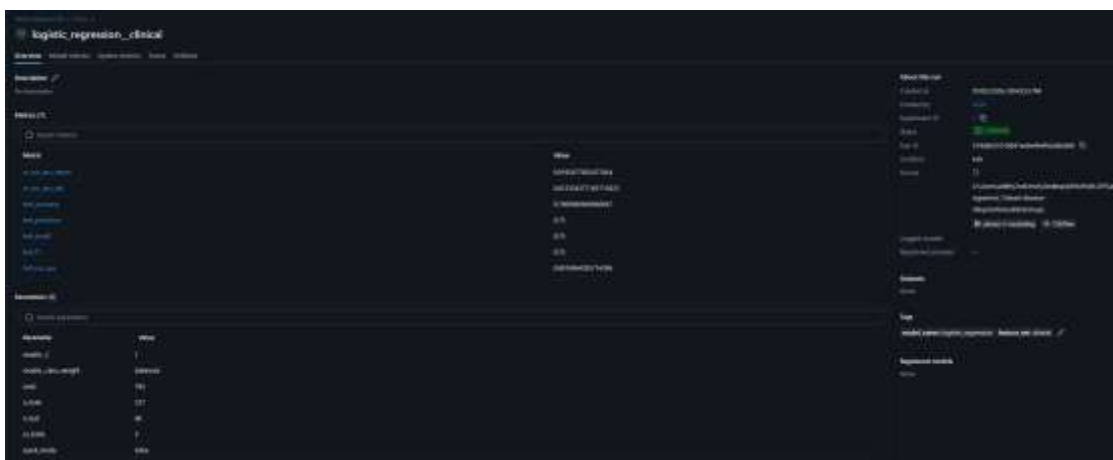


Figure 8 - Best run (LR/clinical) overview with all logged params and metrics

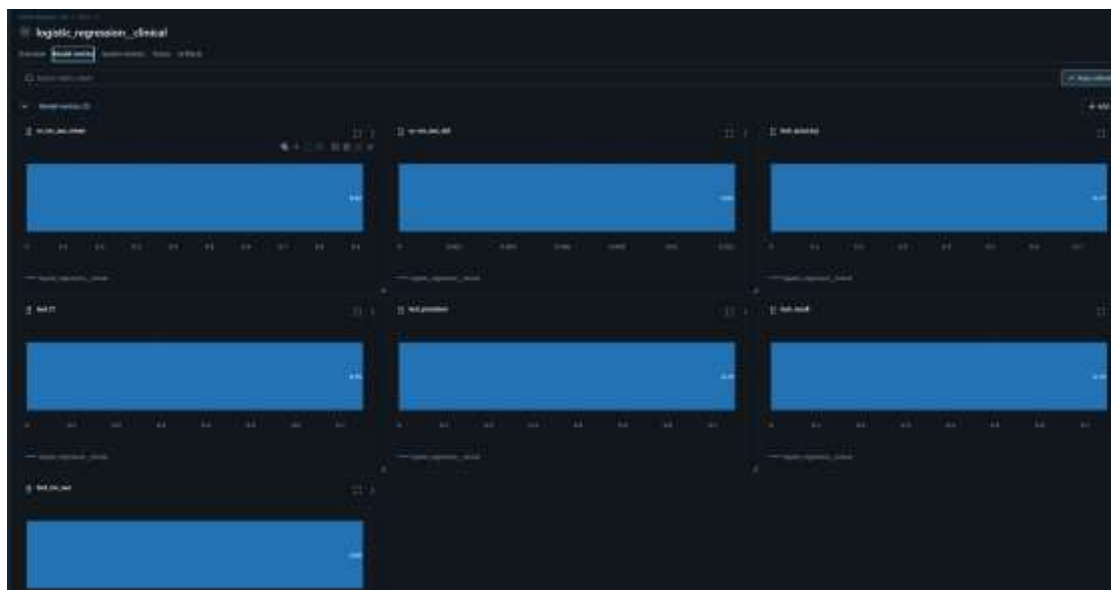


Figure 9 - Best run metric time-series charts

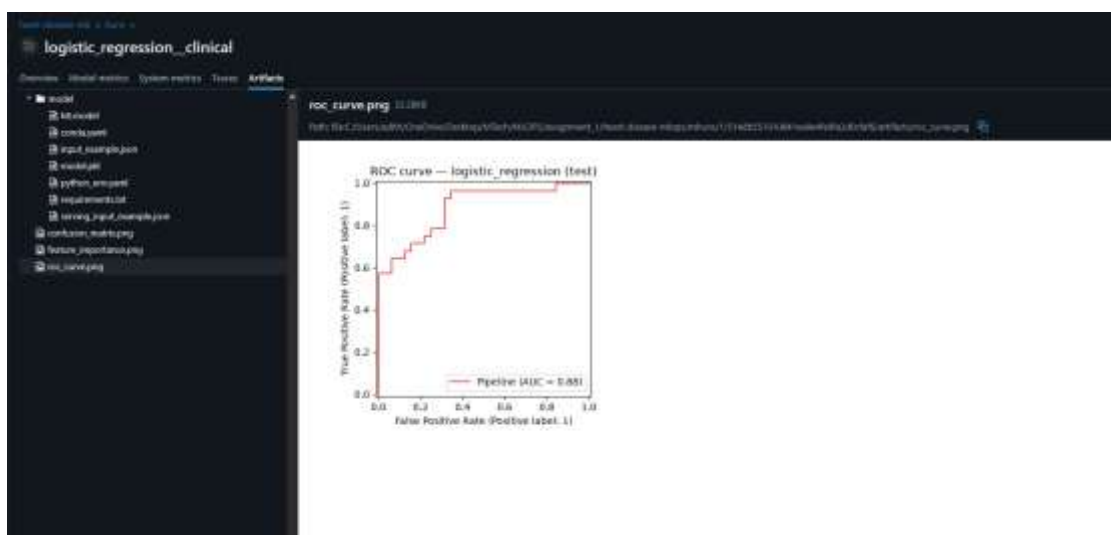
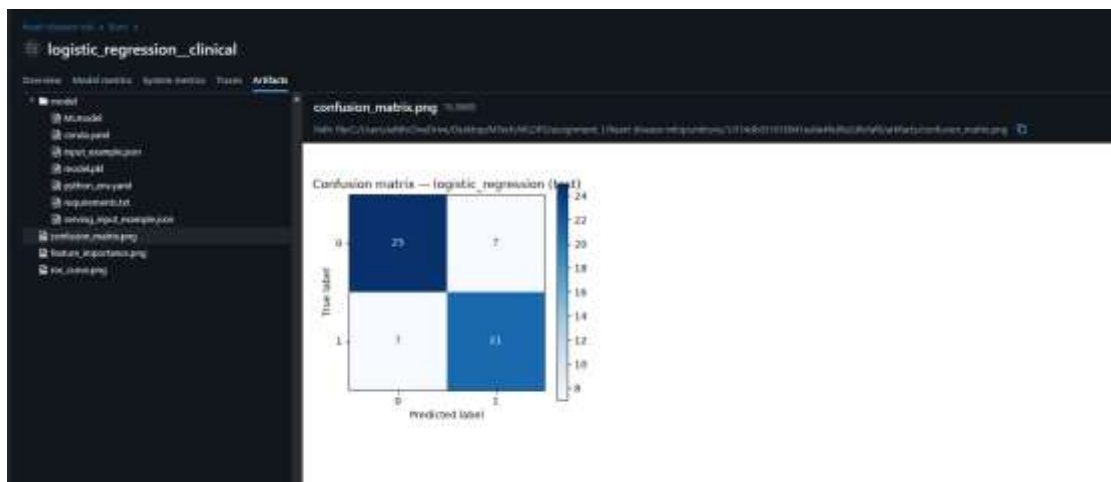
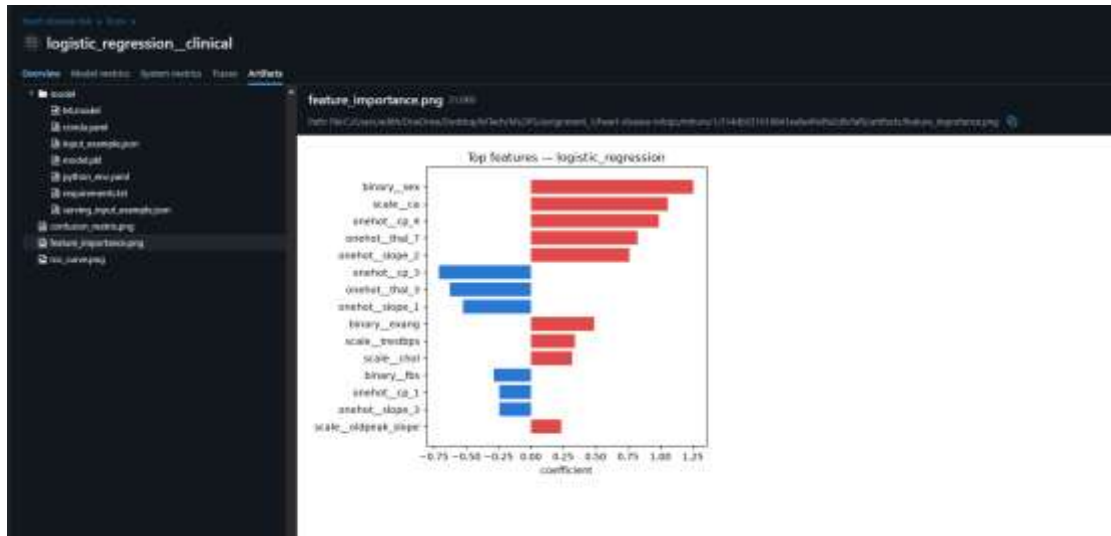


Figure 10 - ROC curve artifact (test split)



## MLflow 3.14 Adaptations

Three deviations from a literal transcription of the assignment brief were required by the pinned MLflow 3.14.0:

- File store deprecation: MLflow 3.x raises on any file:// tracking URI unless `MLFLOW_ALLOW_FILE_STORE=true` is set. Fix: point at `sqlite:///mlflow.db` for metadata.
- `log_model()` changed: `mlflow.sklearn.log_model()` now records the model as a decoupled `LoggedModel` entity outside the run's artifact tree. Fix: `save_model()` to a temp dir + `log_artifacts()` to restore the classic runs: `/id/model` path.
- skops serialisation: the `FunctionTransformer` wrapping `add_clinical_features` is rejected as an 'untrusted type'. Fix: `serialization format=SERIALIZATION_FORMAT_CLOUDPICKLE`.

## 5. Packaging & Reproducibility

The best run (argmax test ROC-AUC over full-grid runs) is exported by `python -m ml.models.export` to `models/` in three formats:

- `heart_disease_pipeline.joblib` - canonical serving artifact; loads with scikit-learn + joblib only (no MLflow at inference time)
- `model_metadata.json` - source run ID, metrics, package versions, and input schema for provenance without the MLflow store
- `mlflow_model/` - same pipeline in MLflow model format

All three are committed to git. A fresh clone can serve the model without retraining. Pickle version-fragility is mitigated by pinned requirements and explicit package version recording in the metadata.

`Notebooks/03_inference.ipynb` demonstrates this standalone: it loads the committed joblib directly - no API server, no MLflow - prints the exported model's provenance from the metadata file, scores the sample request from `api/sample_request.json` (prediction 0,  $P(\text{disease}) \approx 0.33$ , matching the API's response for the same record), and runs batch inference over the full cleaned dataset. This is the direct evidence that the preprocessing pipeline is reusable at inference time.

Installing MLflow 3.14.0 forced pandas from 3.0.3 → 2.3.3 (MLflow declares `Requires-Dist: pandas<3`). This conflict was confirmed with `pip install --dry-run` and the full test suite was re-run against the downgraded pin before committing it.

## 6. CI/CD Pipeline

The GitHub Actions pipeline (.github/workflows/ci.yml) runs four jobs on every push and pull request. lint and test run in parallel; train-smoke and docker depend on test passing:

| Job         | Description                                                                                                                                 |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| lint        | ruff check . - pycodestyle errors (E), pyflakes (F), import order (I). Fails on any finding.                                                |
| test        | Full pytest suite (39 tests). JUnit XML uploaded as artifact even on failure (if: always()).                                                |
| train-smoke | python -m ml.models.train --quick - all 6 model/feature combinations with quick grids. Training log + MLflow DB uploaded as artifacts.      |
| docker      | docker build + docker run + curl /health + curl /predict smoke test. Container must answer a real prediction request for the build to pass. |

Python is pinned to 3.13 across all CI jobs because the committed model pickle was built under 3.13.11 - cross-minor-version unpickling is the most realistic breakage vector. The smoke job does not run export: pick\_best() rejects quick-mode runs by design, and carving out a CI-only bypass would weaken that guarantee; export correctness is covered by unit tests in the test job.

To prove the gate is real, a deliberately broken commit was pushed. The test job fails, downstream jobs are skipped, and the failing assertion is visible in the logs.

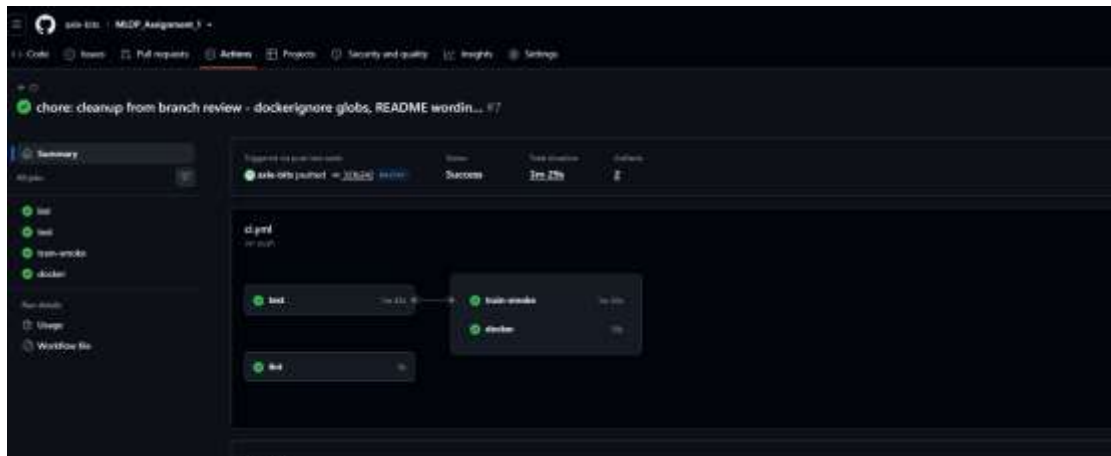


Figure 13 - Green CI run: all four jobs pass

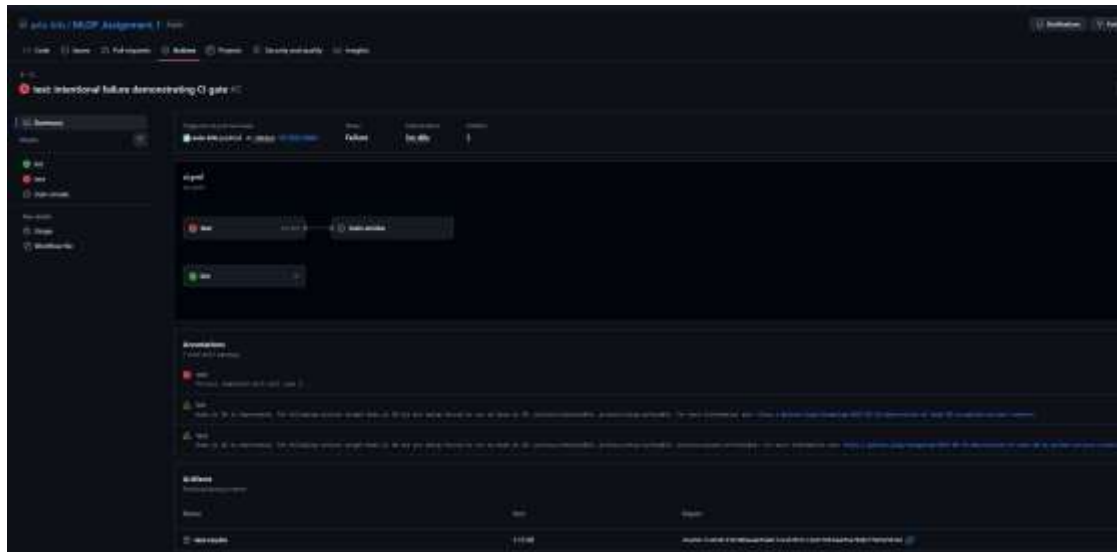


Figure 14 - Red CI run: test failure skips downstream jobs

## 7. Containerisation

### Image Design

The serving image installs only api/requirements.txt - eight pins:

- fastapi==0.139.0
- uvicorn==0.49.0
- scikit-learn==1.9.0
- pandas==2.3.3
- numpy==2.5.0
- joblib==1.5.3
- prometheus\_client==0.25.0
- prometheus-fastapi-instrumentator==8.0.2

MLflow, XGBoost, and the plotting stack are excluded - the exported joblib pipeline needs only scikit-learn, pandas, and the ml package at inference time. This roughly halves image size vs installing full training requirements and shrinks the attack surface.

Additional image properties:

- Base: python:3.13-slim (matches the pickle's build Python version)
- Non-root user (appuser) - principle of least privilege
- HEALTHCHECK against /health (30s interval, 3s timeout, 10s start-period)
- Dockerfile layers ordered: requirements → ml/ → models/ → api/ (cache-optimal)

### Local Proof

The image was built and run with `docker run -d -p 8000:8000 heart-disease-api`. `/health` returned `{"status":"ok","model_loaded":true}` and a `/predict` call on the sample request returned probability 0.3326, confirmed against the container logs.



Figure 15 - Swagger UI served from the container



## API Input Validation

The /predict schema enforces the UCI data dictionary strictly at the API boundary:

- Categorical codes as strict enums: cp  $\in \{1,2,3,4\}$ , thal  $\in \{3,6,7\}$ , ca  $\in \{0,1,2,3\}$ , sex/fbs/exang  $\in \{0,1\}$
- Bounded continuous ranges: age [18,100], trestbps [80,220], chol [100,600], thalach [60,220], oldpeak [0,7]
- Invalid input  $\rightarrow$  HTTP 422 Unprocessable Entity (never a silent wrong prediction)

## 8. Deployment

### Kubernetes Manifests

Three manifest files under infra/k8s/ define the full stack:

- 00-namespace.yaml - isolated heart-disease namespace
- deployment.yaml - 2-replica Deployment; imagePullPolicy: IfNotPresent (local image store); readiness + liveness probes on /health; CPU 100m–500m, memory 256Mi–512Mi per pod
- service.yaml - LoadBalancer Service; port 80 → containerPort 8000

### Verification

On a live 2-node cluster (desktop-control-plane + desktop-worker, v1.36.1): both nodes Ready, both pods Running with READY 1/1. curl /health and a /predict call both succeeded through the LoadBalancer. A rolling-restart demo issued six consecutive health-check requests during the restart - all returned 200 - confirming zero-downtime rollout.

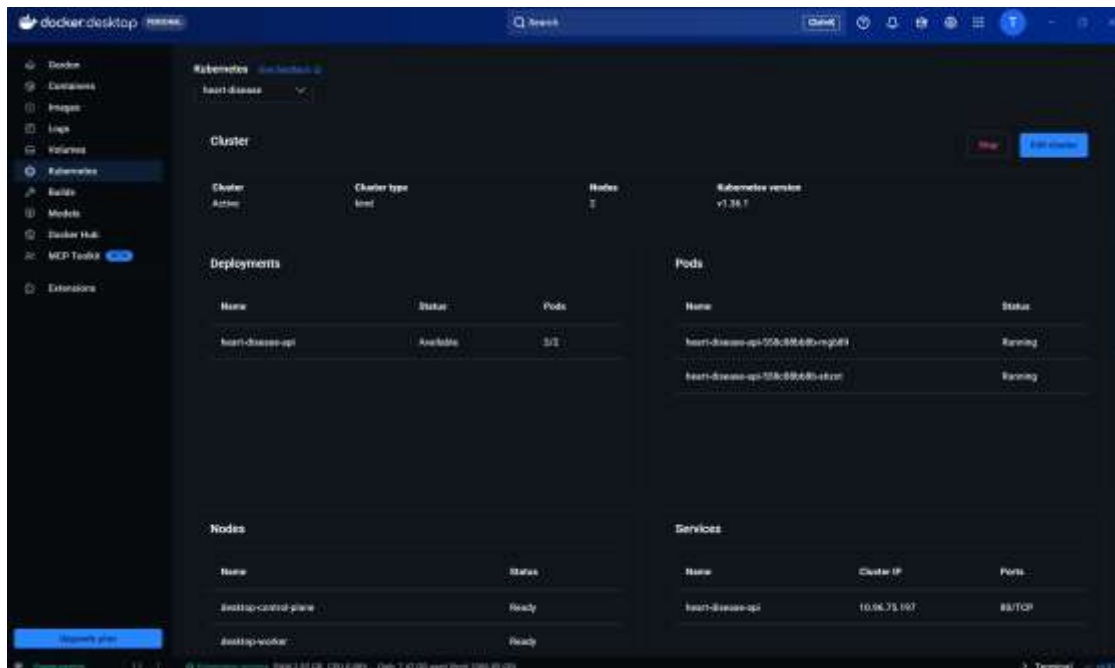


Figure 16 - Docker Desktop Kubernetes cluster dashboard

```
C:\Users\adith\OneDrive\Desktop\WTech\MLOPS\assignment_1\heart-disease-mlops>kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
desktop-control-plane Ready    control-plane   41m   v1.36.1
desktop-worker       Ready    worker         41m   v1.36.1

C:\Users\adith\OneDrive\Desktop\WTech\MLOPS\assignment_1\heart-disease-mlops>kubectl get all -n heart-disease -o wide
NAME                                STATUS    RESTARTS   AGE   IP              NODE           NOMINATED NODE   READINESS GATES
pod/heart-disease-api-556c884db-ngh8f 1/1      Running    0     28m   10.244.1.4     desktop-worker   <none>            <none>
pod/heart-disease-api-556c884db-stcrn 1/1      Running    0     28m   10.244.1.5     desktop-worker   <none>            <none>

NAME                                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE   SELECTOR
service/heart-disease-api            LoadBalancer  10.96.31.197  <none>        80:TCP           27m   app=heart-disease-api

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS   IMAGES                               SELECTOR
deployment.apps/heart-disease-api    2/2     2             2           27m   api          heart-disease-api:v1               app=heart-disease-api

NAME                                DESIRED   CURRENT   READY   AGE   CONTAINERS   IMAGES                               SELECTOR
replicaset.apps/heart-disease-api-556c884db 2         2         2       27m   api          heart-disease-api:v1               app=heart-disease-api,pod-template-hash=556c884db
replicaset.apps/heart-disease-api-547b66522b 0         0         0       0     api          heart-disease-api:v1               app=heart-disease-api,pod-template-hash=547b66522b

C:\Users\adith\OneDrive\Desktop\WTech\MLOPS\assignment_1\heart-disease-mlops>
```

Figure 17 - `kubectl get all -n heart-disease`: 2 pods Running, LoadBalancer active

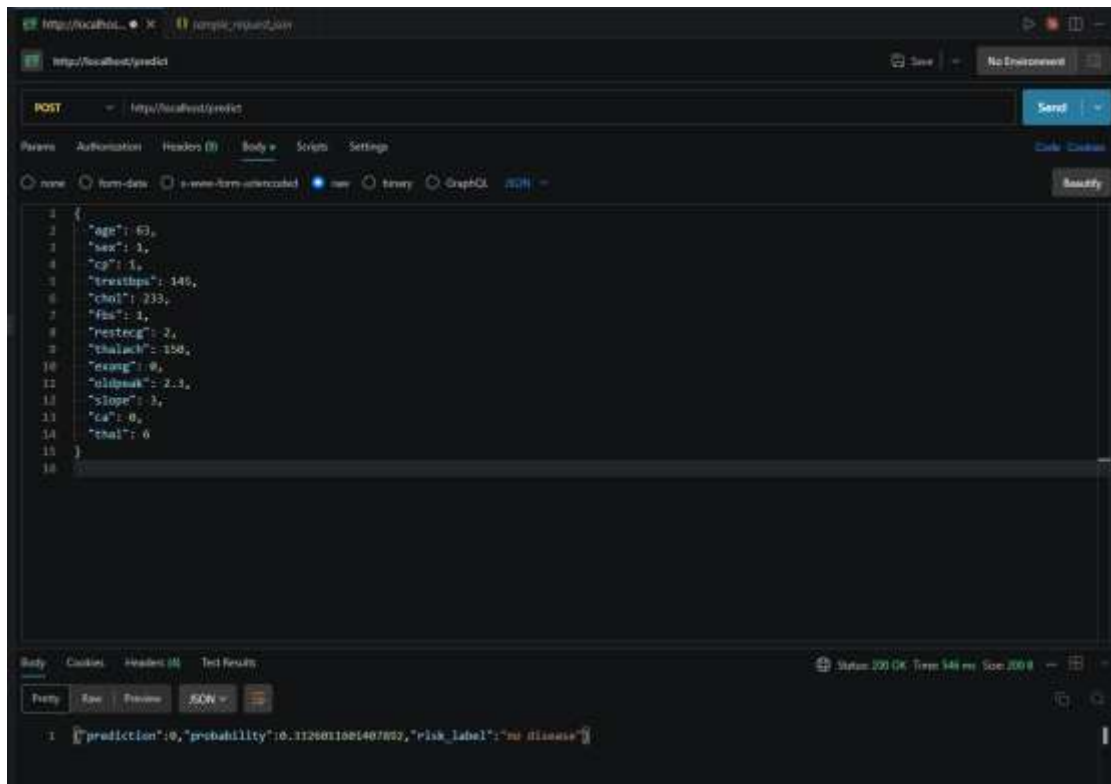


Figure 18 - Postman `/predict` response from the Kubernetes-served API

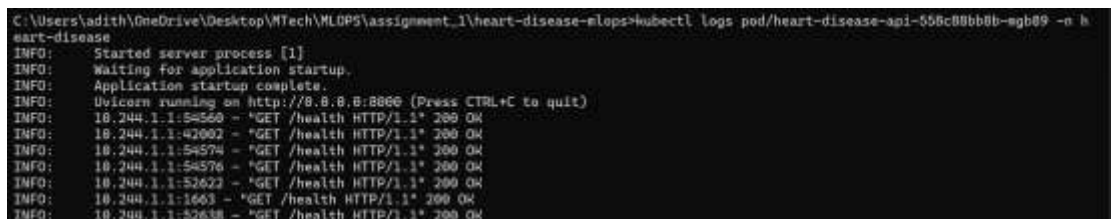


Figure 19 - Pod startup logs: model load + readiness probe hits

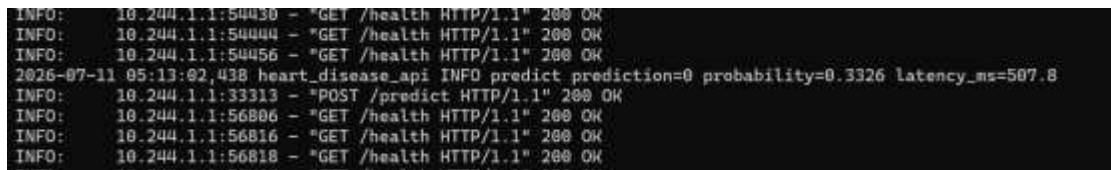


Figure 20 - Structured prediction log line: prediction, probability, latency\_ms

```

C:\Users\adith\OneDrive\Desktop\NTech\MLOPS\assignment_1\heart-disease-nlops>kubectl rollout restart deployment/heart-disease-api -n heart-disease
deployment.apps/heart-disease-api restarted

C:\Users\adith\OneDrive\Desktop\NTech\MLOPS\assignment_1\heart-disease-nlops>kubectl get pods -n heart-disease
NAME                                READY   STATUS    RESTARTS   AGE
heart-disease-api-56886dff69-4fmbb  0/1     Running   0           2s
heart-disease-api-5b75c7bf86-7j2ws  1/1     Running   0           68s
heart-disease-api-5b75c7bf86-dh4wp  1/1     Running   0           75s

C:\Users\adith\OneDrive\Desktop\NTech\MLOPS\assignment_1\heart-disease-nlops>kubectl get pods -n heart-disease
NAME                                READY   STATUS    RESTARTS   AGE
heart-disease-api-56886dff69-4fmbb  0/1     Running   0           4s
heart-disease-api-5b75c7bf86-7j2ws  1/1     Running   0           78s
heart-disease-api-5b75c7bf86-dh4wp  1/1     Running   0           77s

C:\Users\adith\OneDrive\Desktop\NTech\MLOPS\assignment_1\heart-disease-nlops>kubectl get pods -n heart-disease
NAME                                READY   STATUS    RESTARTS   AGE
heart-disease-api-56886dff69-4fmbb  1/1     Running   0           7s
heart-disease-api-56886dff69-ws27z  0/1     Running   0           1s
heart-disease-api-5b75c7bf86-7j2ws  1/1     Terminating   0           73s
heart-disease-api-5b75c7bf86-dh4wp  1/1     Running   0           86s

C:\Users\adith\OneDrive\Desktop\NTech\MLOPS\assignment_1\heart-disease-nlops>kubectl get pods -n heart-disease
NAME                                READY   STATUS    RESTARTS   AGE
heart-disease-api-56886dff69-4fmbb  1/1     Running   0           9s
heart-disease-api-56886dff69-ws27z  0/1     Running   0           3s
heart-disease-api-5b75c7bf86-dh4wp  1/1     Running   0           82s

C:\Users\adith\OneDrive\Desktop\NTech\MLOPS\assignment_1\heart-disease-nlops>kubectl get pods -n heart-disease
NAME                                READY   STATUS    RESTARTS   AGE
heart-disease-api-56886dff69-4fmbb  1/1     Running   0          11s
heart-disease-api-56886dff69-ws27z  0/1     Running   0           5s
heart-disease-api-5b75c7bf86-dh4wp  1/1     Running   0           84s

C:\Users\adith\OneDrive\Desktop\NTech\MLOPS\assignment_1\heart-disease-nlops>

```

Figure 21 - Rolling restart: six consecutive /health 200 OK responses during pod replacement

## 9. Monitoring & Logging

### Structured Logging

Every prediction request emits a structured application log line alongside uvicorn's own access log entry:

```
heart_disease_api INFO predict prediction=<0|1> probability=<p> latency_ms=<t>
```

This gives a human-readable audit trail with stable, greppable fields for every request.

### Prometheus Metrics

The API exposes a /metrics endpoint via prometheus-fastapi-instrumentator, providing:

- Standard HTTP metrics: request rate, latency histogram (p50/p95), non-2xx rate
- Custom domain counter: heart\_disease\_predictions\_total{risk\_label} - incremented on every prediction

### Grafana Dashboard

Prometheus and Grafana are deployed into the same namespace, fully provisioned from ConfigMaps with a single `kubectl apply -f infra/k8s/monitoring/`. The pre-built 'Heart Disease API' dashboard covers request rate, p50/p95 latency, non-2xx rate, and predictions by risk label - both services reachable via LoadBalancer on ports 9090 and 3000.

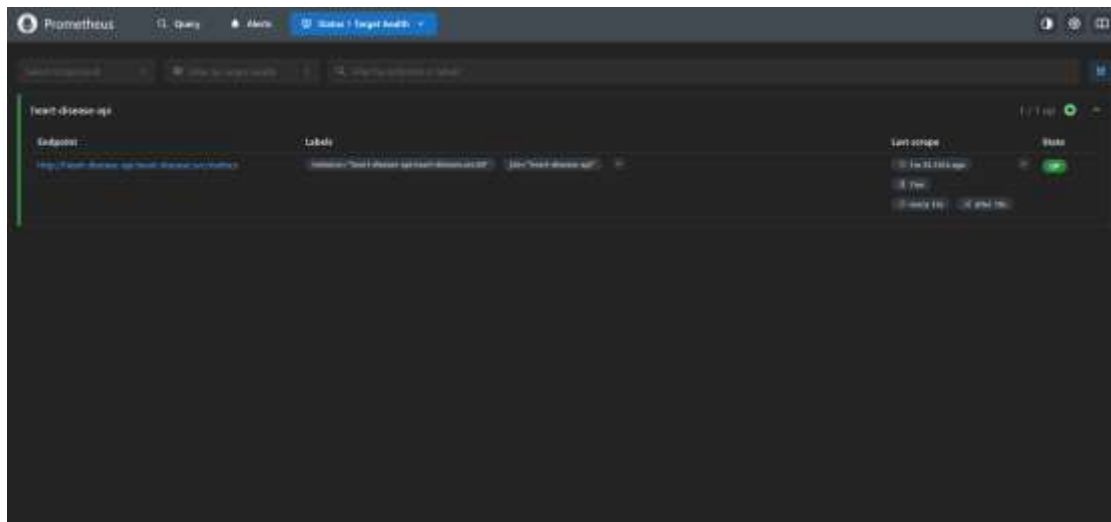


Figure 22 - Prometheus targets page: heart-disease-api scrape target UP



Figure 23 - Grafana 'Heart Disease API' dashboard with live traffic

```

+ - - - - - kubernetes/monitoring
http_response_size_bytes_sum(handler="/predict") 3575.0
# HELP http_response_size_bytes_created Content length of outgoing responses by handler. Only value of header is respected, otherwise ignored. No percentile calculated.
# TYPE http_response_size_bytes_created gauge
http_response_size_bytes_created(handler="/health") 1.7837886888888888
http_response_size_bytes_created(handler="/metrics") 1.7837886888888888
http_response_size_bytes_created(handler="/predict") 1.7837886888888888
# HELP http_request_duration_high_seconds Latency with heavy buckets but no API specific labels. Made for more accurate percentile calculations.
# TYPE http_request_duration_high_seconds histogram
http_request_duration_high_seconds_bucket{le="0.00"} 330.0
http_request_duration_high_seconds_bucket{le="0.005"} 382.0
http_request_duration_high_seconds_bucket{le="0.01"} 392.0
http_request_duration_high_seconds_bucket{le="0.025"} 392.0
http_request_duration_high_seconds_bucket{le="0.05"} 382.0
http_request_duration_high_seconds_bucket{le="0.1"} 382.0
http_request_duration_high_seconds_bucket{le="0.25"} 382.0
http_request_duration_high_seconds_bucket{le="0.5"} 382.0
http_request_duration_high_seconds_bucket{le="1.0"} 382.0
http_request_duration_high_seconds_bucket{le="2.0"} 382.0
http_request_duration_high_seconds_bucket{le="4.0"} 382.0
http_request_duration_high_seconds_bucket{le="8.0"} 382.0
http_request_duration_high_seconds_bucket{le="16.0"} 382.0
http_request_duration_high_seconds_bucket{le="32.0"} 382.0
http_request_duration_high_seconds_bucket{le="64.0"} 382.0
http_request_duration_high_seconds_bucket{le="128.0"} 382.0
http_request_duration_high_seconds_bucket{le="256.0"} 382.0
http_request_duration_high_seconds_bucket{le="512.0"} 382.0
http_request_duration_high_seconds_bucket{le="1024.0"} 382.0
http_request_duration_high_seconds_bucket{le="2048.0"} 382.0
http_request_duration_high_seconds_bucket{le="4096.0"} 382.0
http_request_duration_high_seconds_bucket{le="8192.0"} 382.0
http_request_duration_high_seconds_bucket{le="16384.0"} 382.0
http_request_duration_high_seconds_bucket{le="32768.0"} 382.0
http_request_duration_high_seconds_count 392.0
http_request_duration_high_seconds_sum 0.0000000000000000
# HELP http_request_duration_high_seconds_created Latency with heavy buckets but no API specific labels. Made for more accurate percentile calculations.
# TYPE http_request_duration_high_seconds_created gauge
http_request_duration_high_seconds_created 1.7837886888888888
# HELP http_request_duration_seconds Latency with only few buckets by handler. Made to be only used if aggregation by handler is important.
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket(handler="/health",le="0.5",method="GET") 334.0
http_request_duration_seconds_bucket(handler="/health",le="0.5",method="GET") 334.0
http_request_duration_seconds_bucket(handler="/metrics",le="1.0",method="GET") 334.0
http_request_duration_seconds_bucket(handler="/metrics",le="1.0",method="GET") 334.0
http_request_duration_seconds_count(handler="/health",method="GET") 334.0
http_request_duration_seconds_sum(handler="/health",method="GET") 0.0000000000000000
http_request_duration_seconds_bucket(handler="/metrics",le="0.5",method="GET") 334.0
http_request_duration_seconds_bucket(handler="/metrics",le="0.5",method="GET") 334.0
http_request_duration_seconds_bucket(handler="/predict",le="1.0",method="POST") 334.0
http_request_duration_seconds_bucket(handler="/predict",le="1.0",method="POST") 334.0
http_request_duration_seconds_count(handler="/metrics",method="GET") 334.0
http_request_duration_seconds_sum(handler="/metrics",method="GET") 0.0000000000000000
http_request_duration_seconds_bucket(handler="/predict",le="0.5",method="POST") 334.0
http_request_duration_seconds_bucket(handler="/predict",le="0.5",method="POST") 334.0
http_request_duration_seconds_count(handler="/predict",method="POST") 334.0
http_request_duration_seconds_sum(handler="/predict",method="POST") 0.0000000000000000
# HELP http_request_duration_seconds_created Latency with only few buckets by handler. Made to be only used if aggregation by handler is important.
# TYPE http_request_duration_seconds_created gauge
http_request_duration_seconds_created(handler="/health",method="GET") 1.7837886888888888
http_request_duration_seconds_created(handler="/metrics",method="GET") 1.7837886888888888
http_request_duration_seconds_created(handler="/predict",method="POST") 1.7837886888888888

```

Figure 24 - /metrics endpoint in browser: Prometheus text format

## Observability Lesson: Scrape Topology

Watching the dashboard during a longer traffic session exposed a real observability lesson. Prometheus initially scraped the API through its load-balancing Service, so each scrape landed on a different pod and returned that pod's independent counter. The series jumped between two values, Prometheus read every downward jump as a counter reset, and the rate panels inflated far beyond the true request

volume. The fix was to scrape each pod directly: a headless Service plus DNS-based service discovery gives Prometheus one target per pod, per-pod counters stay monotonic, and the summed panels now report true totals. The takeaway is that scrape topology is part of metric correctness, not just plumbing.

## 10. Conclusion

The pipeline demonstrates the full lifecycle requested by the assignment:

- Data fetched and cleaned reproducibly via a pip-installable download script
- Feature engineering embedded in a pipeline - identical transformation at training and inference
- Tracked multi-model comparison reporting honest, mixed results (clinical features helped LR, hurt Random Forest)
- Deterministic packaging - fresh clone serves without retraining
- CI/CD gate proven with both a green run and a deliberate red run
- Slim Docker image running as non-root with strict API input validation
- Kubernetes deployment chosen for a documented account constraint, not convenience
- Real Prometheus + Grafana monitoring stack wired end-to-end (not a stub endpoint)
- 39-unit tests covering data prep, feature engineering, export logic, and API behaviour